Tugas Besar IF4070
Representasi Pengetahuan dan Penalaran

Milestone 1 Part 2

Translasi Basis Pengetahuan OWL ke SWI-Prolog

Kelompok	K11
Domain	Mahjong *riichi* (WRC Rules 2025, *house rules* klub)

Nama	**NIM**
Ahmad Ibrahim	13523089
Bintang Hijriawan Jachja	13521003
Michael Alexander Angkawijaya	13523102

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

2026

1 Pendahuluan

Part 1 menghasilkan ontologi OWL untuk domain mahjong empat pemain varian klub: 45 kelas, 19 aturan domain, dan 12 *competency question* (CQ). Part 2 meminta ontologi itu dijalankan sebagai program: basis pengetahuan dalam SWI-Prolog, dengan antarmuka `classify/2`, `explain/2`, dan `why_not/2`.

Perbedaan mendasar antara keduanya bukan bahasa, melainkan asumsi. OWL menganut *open world*: pernyataan yang tidak ada dianggap belum diketahui. Prolog menganut *closed world*: pernyataan yang tidak dapat dibuktikan dianggap salah. Karena itu sistem ini tidak sekadar menyalin aksioma; ia harus menjelaskan akibat dari perubahan asumsi itu, dan itu yang dibahas pada Bagian 4.

Sistem terdiri atas tiga berkas: `kb.pl` berisi 51 klausa aturan tanpa satu pun triple individu, `abox.pl` berisi 260 triple untuk 52 individu, dan `main.pl` sebagai titik masuk. Fakta individu pada kasus uji disuntikkan saat runtime, sehingga `kb.pl` dapat dimuat sendirian — bentuk yang dipakai asisten saat menguji.

2 Pemetaan OWL/DL ke Prolog

Entitas ontologi dipetakan satu-ke-satu menjadi predikat, dengan konvensi nama *snake case*: kelas `WinningHand` menjadi `winning_hand/1`, properti `hasYaku` menjadi `has_yaku/2` di dalam `triple/3`. Aksioma menjadi klausa.

Tabel 1: Pemetaan aksioma Part 1 ke klausa Prolog

Aksioma DL (Part 1)	Klausa Prolog (Part 2)
WinningHand $\sqsubseteq$ ValidHand	`aturan(r10, winning_hand(H), (valid_hand(H), yaku_tangan(H,_))).`
WinningHand $\equiv$ ValidHand $\sqcap$ $\exists$hasYaku.Yaku	`yaku_tangan/2` punya tiga klausa: yaku eksplisit (r10), tanyao (r12), yakuhai (r13).
ValidHand $\equiv$ PlayerHand $\sqcap$ $\geq$4 hasGroup $\sqcap$ $\geq$1 hasPair	`aturan(r9, valid_hand(H), (player_hand(H), group_count(H,4), pair_count(H,1))).`
Tanyao $\sqsubseteq$ Yaku	`aturan(r12, yaku_tangan(H, y_tanyao), (tanyao_pattern(H), tanyao_bernilai(H))).`
Individual dan *property assertion*	`triple(Subjek, Predikat, Objek)` di `abox.pl` atau pada kasus uji.

Kelas dengan banyak klausa tidak dipecah menjadi beberapa predikat. Aturan r10 misalnya punya dua klausa untuk `winning_hand/1`: satu menuntut yaku eksplisit, satu menerima tanyao sebagai yaku. ID aturan dipertahankan sama dengan Part 1, sesuai ketentuan bahwa setiap aturan domain harus dapat ditelusuri ke nomornya.

3 Implementasi aturan non-OWL

Lima aturan pada Part 1 tidak dapat dinyatakan murni dalam OWL. Berikut bentuk Prolog-nya dan alasan teknisnya.

R5 — enumerasi urutan *sequence*. OWL tidak punya tipe daftar; urutan tile tidak dapat dinyatakan sebagai aksioma. Di Prolog, `sequence/1` dapat diperiksa dari data tile, tetapi

pada sistem ini pemeriksaan dibatasi pada tipe group karena `abox.pl` tidak menyimpan urutan eksplisit:

```
aturan(r5, sequence(G), triple(G, rdf_type, sequence)).
```

Keterbatasan ini dicatat pada Bagian 7.

R11 — ketersediaan tile untuk tenpai. Menunggu tile yang keempat salinannya sudah dipegang sendiri bukan tenpai sah. Ini aturan kardinalitas atas dunia, bukan atas individu, sehingga butuh aritmetika:

```
aturan(r11, tenpai_hand(H),
       ( player_hand(H), tile_count(H, 13),
         triple(H, waiting_for, W), tile_masih_ada(H, W) )).
aturan(r11, tile_masih_ada(H, W), ( copies_held(H, W, N), N < 4 )).
```

R16 — penjumlahan han. OWL tidak punya penjumlahan. Prolog memakai `sum_list/2`:

```
aturan(r16, total_han(H, Total),
       ( winning_hand(H),
         findall(V, (triple(H, has_yaku, Y), triple(Y, han_value, V)), NV),
         sum_list(NV, HYaku), jumlah_dora(H, HD), Total is HYaku + HD )).
```

R18 — perbandingan skor meja. Syarat "pemain dengan skor terendah" adalah perbandingan antar-individu yang tidak tertutup, sehingga OWL hanya bisa menyatakannya dengan aksioma bantu:

```
aturan(r18, skor_meja_terendah(P),
       ( triple(P, skor_meja, S), triple(P, di_meja, M),
         \+ ( triple(Q, di_meja, M), Q \== P,
              triple(Q, skor_meja, SQ), SQ < S ) )).
```

R19 — override house rules. Tanyao pada tangan terbuka hanya bernilai bila tangan dinilai menurut WRC. Ini konflik prioritas antar aturan, yang di OWL tidak bisa membatalkan inferensi lain (monoton):

```
aturan(r19, tanyao_bernilai(H), closed_hand(H)).
aturan(r19, tanyao_bernilai(H), triple(H, ruleset, wrc)).
```

4 Analisis *open world* versus *closed world*

CQ6 menjadi contoh paling jelas. Pertanyaannya: apakah tangan dengan empat group dan satu pair tanpa yaku tergolong menang? Tangan yang diuji adalah 123m (meld) + 456m + 789p + 888s + 55p — sah secara bentuk, mengandung terminal, satu triplet, tanpa yakuhai.

4.1 Perilaku di OWL

Aksioma Part 1 menyatakan WinningHand $\equiv$ ValidHand $\sqcap$ $\exists$hasYaku.Yaku. Individu `tangan_tanpa_yaku` di-assert sebagai `ValidHand` tetapi tidak memiliki satu pun triple `hasYaku`. Di bawah *open world*, ketiadaan itu tidak berarti tidak ada: bisa saja yakunya ada tetapi belum dicatat. Karena itu HermiT tetap menyatakan ontologi *konsisten*, dan `tangan_tanpa_yaku`

tidak dapat diklasifikasikan sebagai `WinningHand`. Jawaban Protégé bukan "tidak", melainkan "tidak terbukti".

Untuk memperoleh negasi yang tegas di OWL diperlukan aksioma penutup tambahan, misalnya menyatakan kelas `TanpaYaku` secara eksplisit dan meng-assert individunya ke sana. Tanpa itu, OWL memang tidak dirancang memberi jawaban negatif.

4.2 Perilaku di Prolog

Prolog memakai *negation as failure*: `\+`/1 berhasil bila tujuannya gagal dibuktikan. Akibatnya CQ6 dijawab tegas **tidak**, disertai daftar alasan:

```
?- why_not(classify(cq6_tangan, winning_hand), A).
A = [ gagal(r10, yaku_tangan(cq6_tangan,_)),
      gagal(r10, triple(cq6_tangan,has_yaku,_)),
      gagal(r12, tanyao_pattern(cq6_tangan)),
      gagal(r12, triple(cq6_tangan,has_yaku,_)),
      gagal(r13, punya_yakuhai(cq6_tangan)),
      gagal(r12, \+ (triple(cq6_tangan,has_tile,_), \+ inside_tile(_))),
      gagal(r13, triple(cq6_tangan_g4_8s_1, rdf_type, dragon_tile)),
      gagal(r13, triple(cq6_tangan_g4_8s_1, has_wind,_)) ].
```

Tabel 2: Perbandingan jawaban CQ6 pada dua sistem

Aspek	Protégé (OWL)	Prolog
Asumsi	open world	closed world
Ketiadaan `hasYaku`	belum diketahui	tidak ada
Jawaban	tidak terbukti menang	tidak menang
Alasan	tidak tersedia	8 entri `gagal/2`
Konsistensi	konsisten	tidak relevan
Untuk jawaban negatif	perlu aksioma penutup	otomatis

5 Contoh `explain/2` dan `why_not/2`

5.1 explain/2 untuk CQ8 — jawaban ya

CQ8: tangan terbuka dengan meld 567m, sisa 234m 345p 678s 55s, tanpa yaku lain. Menurut WRC, apakah menang? Pohon bukti (dipotong pada bagian berulang):

```
?- explain(classify(cq8_tangan, winning_hand), B).
bukti(r10, winning_hand(cq8_tangan), [
  bukti(r9, valid_hand(cq8_tangan), [
    bukti(r1, player_hand(cq8_tangan), [fakta(triple(...,rdf_type,player_hand))]),
    fakta(group_count(cq8_tangan,4)),
    fakta(pair_count(cq8_tangan,1)) ]),
  bukti(r12, yaku_tangan(cq8_tangan,y_tanyao), [
    bukti(r12, tanyao_pattern(cq8_tangan), [
      fakta(tile_count(cq8_tangan,14)),
      fakta(14>=13),
      tidak((triple(cq8_tangan,has_tile,_), \+ inside_tile(_))) ]),
    bukti(r19, tanyao_bernilai(cq8_tangan), [
      fakta(triple(cq8_tangan,ruleset,wrc)) ]) ]) ])
```

Cara membacanya: r10 adalah aturan menang dengan dua syarat, sah dan punya yaku. Syarat pertama dipenuhi r9 (empat group, satu pair). Syarat kedua dipenuhi r12 dengan dua sub-syarat:

polanya tanyao (`tile_count` 14, tidak ada tile di luar 2–8), dan tanyao itu bernilai karena r19 menemukan `ruleset wrc`. Tanpa triple terakhir, jawabannya berubah menjadi tidak — inilah titik temu antara domain dan aturan house rules.

5.2 explain/2 untuk CQ10 — jawaban ya

Tangan `tangan_yakuhai_mangan` memiliki triplet yakuhai dan total han 5, sehingga menang lewat r10 klausa pertama dengan `yaku_tangan` eksplisit, dan kategorinya mangan lewat r17. Pohon buktinya lebih pendek karena tidak perlu melewati pola tanyao.

5.3 why_not/2 untuk CQ6 — jawaban tidak

Dibahas pada Bagian 4. Entri pertama (`gagal(r10, yaku_tangan(...))`) sudah cukup untuk menjawab; tujuh entri sisanya adalah penelusuran turunan yang menjelaskan mengapa tiap jalur yaku gagal.

5.4 why_not/2 untuk CQ5 — jawaban tidak

Tangan `tangan_terbuka` tidak memiliki pair, sehingga `gagal(r9, ...)` muncul sebagai alasan utama. Contoh ini menunjukkan bahwa `why_not/2` tidak selalu berhenti di aturan yang sama: kalau kegagalannya struktural, r9 yang dilaporkan; kalau kegagalannya pada yaku, r10 yang dilaporkan.

6 Hasil pengujian

Runner `test/run_tests.pl` membandingkan hasil setiap kasus dengan field `harapan` pada JSON. Setiap kasus menyuntikkan faktanya sendiri lalu membersihkannya kembali, sehingga urutan pengujian tidak memengaruhi hasil.

Tabel 3: Rekapitulasi kasus uji per kategori

Kategori	Jumlah	Contoh yang diuji
umum	6	tipe tangan, terminal, pair
batas	10	rank 2 dan 9, jumlah group kurang satu
pengecualian	11	CQ6 tanpa yaku, CQ2b tile terminal bukan inside
individu baru	3	tangan dan tile yang tidak ada di `abox.pl`
info tak lengkap	3	tangan tanpa data tile, tanpa pair
rantai	5	CQ8 dua tingkat (r10 → r9 + r12)
Total	38	**38/38 lulus, 0 gagal**

Jumlah 38 berasal dari 16 kasus CQ ditambah 22 kasus pada `tests.json`. Seluruh kasus yang diharapkan bernilai salah (`harapan: false`) juga lulus, termasuk lima kasus `info_tak_lengkap` yang menguji perilaku saat data tidak lengkap — kondisi yang paling mungkin muncul saat asisten menguji dengan individu baru.

Waktu eksekusi seluruh 16 kasus CQ di bawah 0,1 detik, jauh di bawah batas 5 detik per kueri. `kb.pl` juga diuji dimuat sendirian: tidak ada error, tidak ada permintaan input, dan tidak ada triple individu yang ikut termuat.

7 Keterbatasan

1. R5 belum memvalidasi urutan tile. Pemeriksaan berhenti pada tipe group, karena `abox.pl` menyimpan rank dan suit tetapi tidak menyimpan posisi dalam sequence. Akibatnya sequence 8-9-1 (yang tidak sah) tidak dapat ditolak sistem ini.
2. Sebagian kasus tidak mencantumkan data tile lengkap. Untuk kasus seperti itu, kegagalan tanyao terjadi karena hitungan tile nol, bukan karena ada tile terminal. Alasan yang dilaporkan tetap benar, tetapi bukan alasan "sebenarnya" menurut domain.
3. Perhitungan han berhenti pada kategori (mangan sampai yonbaiman), belum sampai nilai poin dan pembayaran. R17 hanya memetakan total han ke nama kategori.
4. R19 hanya menangani tanyao. Override house rules untuk yaku lain belum dimodelkan.
5. Tidak ada antarmuka pengguna. Semua interaksi lewat REPL SWI-Prolog.

8 Perubahan dari Part 1

1. Individu tidak lagi seluruhnya berada di `abox.pl`; kasus uji membawa faktanya sendiri. Ini memenuhi syarat bahwa `kb.pl` harus dapat diuji tanpa ABox tetap.
2. Tanyao dipecah menjadi dua predikat: `tanyao_pattern/1` (bentuk tangan) dan `yaku_tangan/2` (nilai yaku). Di Part 1 keduanya satu kelas.
3. Ditambahkan penanda `ruleset` (wrc atau house rules) agar R19 dapat membedakan penilaian, sesuai pertanyaan CQ8 yang eksplisit menyebut WRC.
4. Ditambahkan `seven_pairs_hand/1` dan `thirteen_orphans_hand/1` sebagai alternatif bentuk sah pada r9.
5. ID aturan r1–r19 dipertahankan, dan setiap klausa tetap dapat ditelusuri ke aksioma asalnya di ontologi.

9 Pembagian tanggung jawab

Nama	Kontribusi
Ahmad Ibrahim (13523089)	Meta-interpreter dan antarmuka, runner pengujian, pemetaan aturan ke Prolog, penulisan laporan.
Michael Alexander Angkawijaya (13523102)	Perluasan aturan domain (yaku, kategori han, seven pairs, thirteen orphans), model `ruleset`, kasus uji mandiri.
Bintang Hijriawan Jachja (13521003)	Tidak berkontribusi pada Part 2.

10 Log penggunaan AI

Penggunaan AI generatif dibatasi pada dua hal: memahami pesan galat SWI-Prolog dan menyusun README. Keduanya dideklarasikan pada tabel berikut.

Alat	Digunakan untuk	Verifikasi
AI generatif (Hermes Agent)	Menafsirkan pesan galat SWI-Prolog saat memuat `kb.pl` dan menjalankan kueri.	Pesan galat dibaca ulang setelah perbaikan, lalu program dijalankan kembali sampai tidak ada galat.
AI generatif (Hermes Agent)	Menyusun dan merapikan teks `README.md`.	Isi README dicocokkan dengan perintah yang benar-benar dijalankan pada terminal.

Di luar dua hal tersebut, seluruh isi tugas disusun oleh anggota kelompok tanpa bantuan AI generatif: penulisan `aturan/3` untuk r1–r19, meta-interpreter dan pembangunan pohon bukti, predikat kontrak (`classify/2`, `explain/2`, `why_not/2`), kasus uji pada `cq.json` dan `tests.json`, serta seluruh analisis pada laporan ini.

11 Referensi

1. World Riichi Championship. *WRC Rules 2025*. Dokumen sumber aturan domain pada Part 1.
2. SWI-Prolog. *SWI-Prolog Reference Manual*, versi 10.
3. W3C OWL Working Group. *OWL 2 Web Ontology Language Document Overview*, edisi kedua.
4. Protégé. *Protégé 5.6 Documentation*, Stanford Center for Biomedical Informatics Research.

A Daftar aturan final

ID	Kepala	Makna
r1	`player_hand/1`	Tangan milik pemain
r2	`terminal_tile/1`	Tile berank 1 atau 9
r3	`inside_tile/1`	Tile berank 2 sampai 8
r4	`honour_tile/1`	Tile honour (wind, dragon)
r5	`sequence/1`	Group berjenis sequence
r6	`quad/1`	Group berjenis quad
r7	`pair/1`	Group berjenis pair
r8	`closed_hand/1`, `open_hand/1`	Ada group meld atau tidak
r9	`valid_hand/1`	Empat group dan satu pair, seven pairs, thirteen orphans
r10	`winning_hand/1`, `yaku_tangan/2`	Sah dan punya yaku
r11	`tenpai_hand/1`	13 tile, menunggu, tile masih tersedia

ID	Kepala	Makna
r12	`tanyao/1`, `tanyao_pattern/1`	Seluruh tile inside
r13	`yakuhai/1`, `punya_yakuhai/1`	Triplet dragon atau wind
r14	`dora_bukan_yaku/1`	Dora menambah han, bukan yaku
r15	`tambahan_han_dora/2`	Tambahan han dari dora
r16	`total_han/2`	Penjumlahan han yaku dan dora
r17	`kategori_han/2`	Mangan sampai yonbaiman
r18	`bonus_comeback/1`	Bonus skor terendah (house rules)
r19	`tanyao_bernilai/1`	House rules membatalkan tanyao terbuka